

🎧 Harmonix — Guía de Ejecución, Integración y Pruebas

Discord Music Bot Hub — Simulador interactivo + Código listo para self-hosting

Si ves este archivo desde la interfaz web, busca GUIA-EJECUCION.md en el explorador de archivos.

Índice

1. ¿Qué es Harmonix?
2. Ejecutar el simulador web (local)
3. Integrar Gemini AI (opcional)
4. Self-hosting: el bot real de Discord
5. Probar antes de producción
6. Solución de problemas comunes
7. Checklist pre-producción

1. ¿Qué es Harmonix?

Harmonix tiene **dos caras**:

Capa	Descripción
Simulador Web (React + Vite + Express)	Interfaz tipo Discord con visualizador de audio, terminal de comandos, letras sincronizadas, y un AI DJ que recomienda música. Ideal para probar y experimentar sin tocar un bot real.
Bot Code Hub	Código fuente real de un bot de Discord.js v14 (con @discordjs/voice y play-dl) que podés copiar, configurar y deployar en tu propio servidor de Discord .

Todo corre en TypeScript. El server.ts orquesta Express + Vite middleware + Gemini AI.

2. Ejecutar el simulador web (local)

Prerrequisitos

- **Node.js** v18+ (recomendado v20 o v22)
- **npm** (viene con Node.js)
- **(Opcional)** Una API Key de [Google Gemini](https://aistudio.google.com/app/apikey) (<https://aistudio.google.com/app/apikey>) para las funciones de IA

Pasos

```
# 1. Parate en la carpeta del proyecto
cd ruta/a/discord-music-bot

# 2. Instalá las dependencias
npm install

# 3. (Opcional) Configurá la API key de Gemini
# Copiá .env.example a .env y poné tu key:
# GEMINI_API_KEY="tu-api-key-aca"

# 4. Levantá el servidor de desarrollo
npm run dev
```

El servidor arranca en **http://localhost:3000** — abrí esa URL en el navegador.

Qué esperar

- Se abre la interfaz con el header **"SYNTH."** y dos tabs: **Interactive Simulator** y **Self-Hosted Bot Code**.
- Del lado izquierdo: servidores de Discord, canales de voz, y lista de reproducción rápida.
- En el centro: visualizador de audio (canvas animado), controles de reproducción, y la terminal de comandos.
- A la derecha: panel de letras sincronizadas o chat con AI DJ.

Comandos básicos del simulador

Escribí en la terminal y presioná Enter:

Comando	Qué hace
/play Neon Odyssey	Busca y reproduce un tema
/skip	Salta al siguiente tema
/stop	Detiene y limpia la cola
/queue	Muestra los temas encolados
/nowplaying	Info del tema actual
/lyrics	Abre las letras sincronizadas
/volume 80	Ajusta el volumen (0-100)
/ai-dj late night coding mood	El AI DJ te recomienda música
/help	Lista completa de comandos

También podés hacer clic en los chips de abajo (**QUICK RUN**) o en los tracks de la lista lateral.

3. Integrar Gemini AI (opcional)

Sin Gemini, el simulador funciona igual pero usa datos locales (6 tracks pre-cargados) y respuestas simuladas. Con Gemini obtenés:

- **Búsqueda inteligente:** pedile /play synthwave chill y Gemini devuelve tracks curados con metadata.
- **Letras generadas:** /lyrics genera letras sincronizadas con timestamps.
- **AI DJ:** describí un mood y recibí recomendaciones narradas como un DJ de Discord.

Configuración

```
# 1. Obtene una API key gratis en https://aistudio.google.com/apikey

# 2. Creá (o editá) el archivo .env en la raíz del proyecto:
GEMINI_API_KEY="AIzaSyTuKeyAcá"
# No le pongas comillas al valor si usás el .env directamente

# 3. Reiniciá el servidor (Ctrl+C y npm run dev de nuevo)
```

Tips: La key se lee del archivo `.env`. Si ves el mensaje `"GEMINI_API_KEY is missing or using placeholder. All features will run in simulation mode"` en la consola, es porque no configuraste la key o todavía tiene el valor `MY_GEMINI_API_KEY`.

4. Self-hosting: el bot real de Discord

El **Bot Code Hub** (segundo tab en la interfaz) contiene el código real de un bot de Discord.js v14. No es una simulación — es código funcional que podés copiar y ejecutar.

Lo que incluye

- `bot.ts` — Cliente Discord con `/play`, `/skip`, `/stop`, cola de reproducción, y streaming de audio.
- `package.json` — Dependencias exactas (`discord.js`, `@discordjs/voice`, `play-dl`, etc.).
- `.env` — Template con `DISCORD_TOKEN`.

Paso a paso para deployar

4.1. Crear la aplicación en Discord

1. Andá a [Discord Developer Portal \(https://discord.com/developers/applications\)](https://discord.com/developers/applications).
2. Hacé clic en **New Application** → ponele nombre (ej. "Harmonix Bot").
3. Andá a la pestaña **Bot** → **Add Bot**.
4. **Copía el Token** — lo vas a necesitar para el `.env`.
5. Activá los **Privileged Gateway Intents**: `MESSAGE CONTENT INTENT`, `SERVER MEMBERS INTENT`, y `GUILD VOICE STATES INTENT`.

4.2. Invitar el bot a tu servidor

1. Andá a la pestaña **OAuth2** → **URL Generator**.
2. Seleccioná los scopes: `bot` y `applications.commands`.
3. En permisos del bot: `Connect`, `Speak`, `Send Messages`, `Use Slash Commands`.
4. Copiá la URL generada, abrila en el navegador, y seleccioná el servidor.

4.3. Instalar dependencias del bot

En tu máquina local o VPS, creá una carpeta nueva para el bot:

```
mkdir harmonix-bot
cd harmonix-bot

# Creá los archivos necesarios (copiá el código del Bot Code Hub)
# bot.ts, package.json, .env, tsconfig.json
```

O copiá el contenido directamente desde el **Bot Code Hub** en la interfaz.

```
# Después de tener los archivos:
npm install
```

El bot necesita **ffmpeg** instalado en el sistema. Si no lo tenés:

- **Windows**: `scoop install ffmpeg` o descargalo de [ffmpeg.org \(https://ffmpeg.org\)](https://ffmpeg.org).
- **macOS**: `brew install ffmpeg`
- **Linux**: `sudo apt install ffmpeg`

4.4. Configurar .env

```
DISCORD_TOKEN="tu-token-del-bot-aca"
```

4.5. Ejecutar el bot

```
npm start
# o si instalaste ts-node globalmente:
ts-node bot.ts
```

Si ves  `Harmonix Bot is logged in as TuBot#1234!`, ya está andando.

Comandos del bot real

- `/play <canción o URL>` — reproducí desde YouTube, Spotify, o SoundCloud.
- `/skip` — saltá al siguiente tema.
- `/stop` — desconectá el bot del canal de voz.

Importante: El bot usa `play-dl` que soporta YouTube, Spotify y SoundCloud. No requiere API keys externas, pero Spotify opcionalmente acepta `SPOTIFY_CLIENT_ID` y `SPOTIFY_CLIENT_SECRET` para mejorar rate limits.

5. Probar antes de producción

Usá este flujo para verificar que todo funciona antes de pensar en deployar a producción.

5.1. Test del Simulador Web

```
# Arrancá el simulador
npm run dev

# Abrió http://localhost:3000
```

Verificá estas 5 cosas:

1. **Visualizador**: se ve el canvas con barras animadas (oscilan aunque no haya audio).
2. **Reproducción local**: hacé clic en un track de la lista lateral → debería empezar a sonar desde el navegador.
3. **Terminal**: escribí `/play Neon Odyssey` → el bot responde con un embed.
4. **Letras**: hacé clic en `/lyrics` → se abre el panel con líneas sincronizadas.
5. **AI DJ**: escribí `/ai-dj música para programar` → recibís una recomendación (simulada si no hay Gemini).

5.2. Test sin Gemini

Si no configuraste Gemini API key, el sistema cae a modos simulados:

- Búsqueda usa los 6 tracks precargados y fuzzy matching sobre título/artista/género.
- Letras usan timestamps simulados genéricos.
- AI DJ responde con un mensaje estático.

Esto está bien para development. No es necesario tener Gemini para probar la UI.

5.3. Test con Gemini

Si configuraste la key:

- Probá búsquedas abstractas: /play algo para estudiar → Gemini debería devolver tracks.
- /lyrics sobre un track de esos → letras generadas por IA.
- /ai-dj dame algo energético → respuesta personalizada.

Si algún endpoint falla, el server loguea el error pero nunca crashea — cae al fallback local.

5.4. Test del bot real de Discord

1. Invisá el bot a un servidor de prueba (no uses el servidor productivo todavía).
2. Ejecutá el bot localmente con `npm start`.
3. Andá a un canal de voz en Discord.
4. Escribí `/play Neon Odyssey` → el bot debería unirse y reproducir.
5. Probá `/skip` y `/stop`.

5.5. Revisión de logs

Revisá el output de la terminal donde corre el bot:

- `server.log` y `server.err` en la raíz del proyecto guardan logs del simulador web.
- El bot real loguea a `stdout` — revisá que no haya errores de conexión o autenticación.

6. Solución de problemas comunes

Problema	Causa probable	Solución
port 3000 already in use	Otro proceso usando el puerto	Cambiá el PORT en <code>server.ts</code> o matá el proceso con <code>npx kill-port 3000</code>
GEMINI_API_KEY is missing	<code>.env</code> no configurado o placeholder	Pone tu API key real en <code>.env</code> , reiniciá el servidor
El visualizador no se ve	El canvas necesita un contenedor visible	Asegurate de que la ventana del navegador tenga suficiente tamaño (al menos 800px de ancho)
El audio no se reproduce	El navegador bloquea autoplay	Hacé clic en el botón de Play manualmente la primera vez
ffmpeg not found	ffmpeg no está instalado	Instalalo y asegurate de que esté en el PATH
DISCORD_TOKEN invalid	Token mal copiado o regenerado	Generá un nuevo token en Discord Developer Portal
El bot no se une al canal	Falta el intent <code>GUILD_VOICE_STATES</code>	Activá el intent en Discord Developer Portal
play-dl falla con YouTube	YouTube cambió algo del lado de ellos	<code>npm update play-dl</code> a la última versión

7. Checklist pre-producción

Usá esto cuando estés listo para pasar a producción.

Simulador Web (si lo vas a hostear)

- [] GEMINI_API_KEY configurada con una key real
- [] NODE_ENV=production en el entorno
- [] Build generado: `npm run build` → produce `dist/`
- [] El servidor Express sirve los archivos estáticos desde `dist/`
- [] Variables de entorno inyectadas por el host (no en el repo)
- [] Puerto configurable via `process.env.PORT`
- [] CORS configurado si aplica
- [] Logs rotados o manejados por el host

Bot real de Discord

- [] Token almacenado como secreto/env var, no hardcodeado
- [] ffmpeg instalado en el entorno productivo
- [] Intentos de reconexión (`Discord.js` lo maneja automáticamente, pero verificá)
- [] El bot solo responde en los servidores que corresponde
- [] Permisos mínimos necesarios (principio de menor privilegio)
- [] Comandos Slash registrados globalmente o por guild (el código los setea globales)
- [] Monitoreo básico (logs, health check)

General

- [] `node_modules` excluido del repo (`.gitignore` ya lo tiene)
- [] `.env` excluido del repo (`.gitignore` ya lo tiene)
- [] Build exitoso sin errores de TypeScript: `npm run lint`
- [] Prueba completa en entorno de staging
- [] Backup del token y API key en un gestor de secrets

¿Preguntas? Revisá el código fuente en `server.ts`, `src/App.tsx`, y el `BotCodeHub.tsx` para entender cada pieza.